

Coding & Solution

Phase 5 — Project Development

The complete source code is in the `pocket smart/` folder next to this file. This document explains how it is organised, how to run it, and how each part works.

1. Folder Structure

- `app.py` — main FastAPI application with all routes.
- `auth.py` — password hashing, tokens, login checks.
- `gemini_utils.py` — Gemini setup, prompts, JSON parsing, shopping links.
- `models.py` — Pydantic data models.
- `requirements.txt` — Python packages.
- `.env.example` — template for environment variables.
- `README.md` — short setup guide.
- `templates/` — eight HTML pages.
- `static/styles.css` — shared styling.
- `static/uploads/` — saved outfit images (created automatically).

2. Setup and Run

Install Python 3 and open a terminal in the `pocketsmart` folder.

Create a virtual environment: `python -m venv venv`.

Activate it. On Windows: `venv\Scripts\activate`. On macOS or Linux: `source venv/bin/activate`.

Install packages: `pip install -r requirements.txt`.

Copy `.env.example` to `.env` and fill in:

- `GOOGLE_API_KEY` — your key from Google AI Studio.
- `SECRET_KEY` — any long random string.

Start the server: `python -m uvicorn app:app --reload`.

Open `http://127.0.0.1:8000` in a browser.

Optional: open `http://127.0.0.1:8000/docs` to see FastAPI's automatic route documentation.

3. Packages Used

`fastapi`, `uvicorn`, `python-dotenv`, `google-generativeai`, `python-jose[cryptography]`, `passlib[bcrypt]`, `python-multipart`, `jinja2`, `pillow`.

4. How the Code Works

4.1 `app.py`

- Creates the FastAPI app titled "PocketSmart: AI Budget Planner".
- Adds CORS middleware, mounts /static, and creates the static/uploads folder if it does not exist.
- history_db is a dictionary that keeps each user's recommendations in memory.
- save_to_history inserts a new record at the start of the user's list with a UUID, a UTC timestamp, the type, the input and the result.
- Public routes: landing page, register page and action, login page and action.
- Register checks for a duplicate username, hashes the password and redirects to login.
- Login verifies credentials, creates a token that lasts 30 minutes and sets it as an HTTP-only cookie.
- Logout adds the token to the blacklist and deletes the cookie.
- Protected routes use Depends(get_current_active_user) so only logged-in users can reach them.
- /home-budget and /party-budget accept JSON validated by Pydantic models.
- /jewelry-budget accepts form fields and an optional image. The image is saved as static/uploads/<uuid><extension> and its path is passed to the AI function.
- /dashboard shows the first five history records; /history shows all.

4.2 auth.py

- Uses CryptContext with bcrypt for hashing and verifying passwords.
- create_access_token adds an expiry time and signs the token with SECRET_KEY using HS256.
- get_token reads the token from the access_token cookie.
- get_current_active_user decodes the token, rejects blacklisted or expired tokens, and returns the stored user or raises a 401 error.

4.3 gemini_utils.py

- Loads .env and stops with a clear error if GOOGLE_API_KEY is missing.
- Configures Gemini and creates the model object.
- extract_json_from_response removes json fences and parses the text; if that fails it uses a regular expression to grab the first { ... } block.
- _add_links goes through each category and item, URL-encodes search_terms and fills the platform URL templates.
- Platform dictionaries: HOME_PLATFORMS (Amazon, Flipkart, IKEA, Myntra), PARTY_PLATFORMS (Amazon, Flipkart, Swiggy, Zomato, BookMyShow), JEWELRY_PLATFORMS (Amazon, Flipkart, Tanishq, CaratLane).
- get_home_recommendations — builds a prompt from the number of lights, fans, furniture and dining tables and the selected rooms, then requests JSON in a fixed structure.
- get_party_recommendations — builds a prompt from party type, guest count, venue and service flags.
- get_jewelry_recommendations — builds a prompt from occasion and preferences; if an image exists, opens it with Pillow and sends it with the prompt so the AI can describe the outfit and match jewelry.

4.4 models.py

Defines the five Pydantic classes so incoming data is checked automatically (for example, a budget must be a number).

4.5 Templates and Styling

- index.html — landing page.
- register.html and login.html — forms that submit with JavaScript fetch.
- dashboard.html — recent recommendations, links to planners and a logout button that calls /logout.
- home_planner.html, party_planner.html, jewelry_planner.html — forms that collect values with FormData, call the matching route and render the returned plan.
- history.html — lists all saved recommendations.
- styles.css — one shared stylesheet of about 150 lines.

5. Prompt Design Notes

- Prompts mention India and INR so prices and stores are relevant.
- Prompts state "Respond ONLY with valid JSON" and show the exact structure, which greatly reduces extra commentary.
- Prompts tell the model to keep the total within budget.
- Optional fields are replaced with "None" or "Not specified" so the prompt never contains empty gaps.

6. Data Flow Example (Home Planner)

User enters budget 100000, 5 lights, 3 fans, 2 furniture pieces, 1 dining table, selects Living Room and Kitchen.

Browser sends JSON to /home-budget.

Server checks login and validates the data.

Prompt is sent to Gemini.

Gemini returns JSON with categories such as lighting, fans, furniture and dining.

Links are added to each item.

The result is stored in history and shown on the page.

7. Known Limitations

- Users, tokens and history are in memory and disappear on restart.
- The code uses the older google.generativeai package and the model name gemini-1.5-flash. If Gemini calls fail, update to the current SDK and a supported model.
- CORS allows all origins.
- If SECRET_KEY is not set, a default value is used — always set your own.
- AI output is not checked afterwards, so a plan might not add up exactly to the budget.
- Uploaded images are never deleted.
- datetime.utcnow() is used for timestamps, which newer Python versions mark as deprecated.

8. Suggested Improvements for Developers

- Add a database layer (SQLModel or SQLAlchemy) for users and history.
- Verify plan totals in code and warn if they exceed the budget.
- Add a retry when the AI returns invalid JSON.

- Validate image type and size before saving.
- Move settings such as the model name and token lifetime into environment variables.
- Add automated tests for routes and the JSON helper.